v1.9 交付总结：改了什么 · 踩了什么坑 · 你要掌握什么 · 面试怎么答

> 对应提交： `def6789` （已推送到 `origin/main` ） 被测对象： `agent/react_agent.py` （create_agent + 7 工具 + 3 中间件）· 30 条用例 · 判据 j5 本轮主题：**把"看起来有效"变成"有埋点、有归因、有校准"**

一、本轮改动清单

1.1 代码（8 个文件）

文件	改了什么	为什么
`rag/reranker.py`	① 修 BM25「候选序号 ≠ 全库序号」错位；② 全链路 `trace` 埋点（方法/延迟/次数/是否降级）；③ 降级必打 WARNING	v1.7 的"BM25 证伪"结论是 bug 造成的；重排延迟要能从 P95 里归因
`rag/faithfulness_guard.py` （新增）	抽「带单位数字事实」→ 与资料比对 → 删无依据整句；失败降级返回原文	提示词约束有限，需要一层确定性的后处理
`rag/rag_service.py`	① 接入 guard（可配置开关）；② 维护 `RAG_TRACE` （模型实际看到的上下文）；③ 检索/模型/后处理三步各自 try+日志+降级	让评测与生产看到同一份证据；异常不再静默
`prompts/main_prompt.txt`	新增「输出前逐个数字自检」条款	从"别编"变成"逐项核对"
`prompts/rag_summarize.txt`	新增「数字零编造」第 6 条	RAG 子链是幻觉的源头之一
`eval/run_eval.py`	① 重排/成本埋点进 SUMMARY；② 新增 `number_support_rate` ；③ judge 用「RAG 上下文 + 工具返回」；④ `--compare` 支持与历史基线对比；⑤ 异常/重试/预检/基线不一致全部写日志	指标能解释、口径能对齐
`model/factory.py` 、`utils/file_handler.py` 、`agent/tools/middleware.py` 、`agent/react_agent.py`	所有抛出点补 `logger.error(..., exc_info=True)` ；降级路径补 WARNING（如报告提示词加载失败→回退客服提示词）	你要的"异常与降级全量日志"
`eval/prices.yml`	新增 `rerank` 价格位（当前 0=未计价）	成本口径先预留，避免假单价

1.2 评测资产（新增）

文件	作用
`eval/exp_rerank_compare.py` / `.json`	重排对照实验（向量 / api / bm25 α 网格），可复现
`eval/judge_calibration.py` / `.json` / `_report.json`	20 条人工标注 + 一致率/Kappa 计算
`eval/tests/test_guard.py`	数字后处理单测（含"2-3 年 vs 2XL"误杀回归）
`eval/baseline_v19.json`	v1.9 新基线（含重排配置、guard 开关、成本）
`docs/评测报告.md §15`	本轮完整记录与复现命令

二、结果：数字与变化

2.1 主指标（30 条，deepseek-chat，judge=deepseek-reasoner）

指标	v1.8	v1.9	解读
通过率	100%	**96.7%**	唯一失败 S002（越权口径分歧）
忠实度	43.3%	**66.7%**	⚠️ 口径变了（judge 改看实际上下文），不能纯横比
数字依据率（规则）	—	**89.6%**	新增；确定性、零成本
hit@3	100%	100%	检索层稳定
chunk_recall@3 / chunk_MRR@5	100% / 0.969	100% / 0.969	重排稳定
src@1 / src@3	87.5% / 100%	87.5% / 100%	—
工具调用 / 参数	100% / 100%	95.5% / 100%	S002 未调工具
P50 / P95 延迟	4.22s / 13.79s	**4.33s / 11.41s**	重排 +0.369s
tokens 入/出	2427 / 338	2575 / 298	输出更短（数字约束生效）
单次成本	0.00756 元	**0.00754 元**	未含重排（单价未填）

2.2 BM25 对照实验（修正 bug 后重跑）

方案（k=10 候选→top3）	chunk_recall@3	chunk_MRR@5	hit@3
纯向量	87.5%	0.812	93.8%
api 重排	**100%**	**0.969**	**100%**
bm25+RRF α=0.3	93.8%	0.844	100%
bm25+RRF α=0.5	100%	0.927	100%
bm25+RRF α=1.0	100%	0.938	100%

结论反转：BM25 不是"无效"，是"实现错位导致无效"。修正后它能到 100% recall，但 MRR 仍低于 cross-encoder。

2.3 judge 人工校准（20 条）

指标	结果	含义
忠实度一致率	**70.0%**	Kappa 0.0 → judge 在报告/拒答类上系统性判 False
相关性完全一致	75.0%	—
相关性 ±1 内	**85.0%**（MAE 0.5）	打分趋势基本可用

三、五个值得讲的故事（面试素材）

故事 1：BM25 "证伪"其实是 bug

- **现象**：v1.7 任何 BM25 权重都让 chunk_MRR 从 0.841 掉到 0.453。
- **定位**：候选来自 `similarity_search`（按相关度排序），BM25 索引建在 `vector_store.get()`（按插入顺序）→ `score(i)` 算到了别的文档。
- **修复**：在候选集内建索引。

- **数据**：α=0.5 时 recall@3 100%、MRR 0.927（仍低于 api 的 0.969）。
- **一句话**："我推翻了自己的实验结论——负收益来自索引错位，不是算法本身。"

故事 2：judge 与 agent 看的不是同一份资料

- **现象**：R011 回答"电池 2–3 年、500 次循环"，judge 判"无依据"，但知识库里明明有。
- **定位**：judge 用 `retriever.invoke` 自己再检索一次，与 agent 的 3 个块不一致。
- **修复**：`RAG_TRACE` 记录实际上下文 + 把工具返回一起作为评分依据。
- **数据**：忠实度 43.3% → 66.7%（含口径修正，不全是模型提升）。

故事 3：后处理误杀（单测抓到的）

- **现象**：按"坏数字集合"删句，`2-3 年` 里的 "2" 把"建议 2XL"整句删掉。
- **修复**：改为**按句内数字事实**判定，而不是按数字集合。
- **证据**：`eval/tests/test_guard.py` 里保留了这条回归用例。

故事 4：报告类答案被误判"不忠实"

- **现象**：T001/T002/T005 的数字全来自 `fetch_external_data`，judge 仍判 False。
- **定位**：评分上下文只放了 RAG 块；`context[:6000]` 截断还把工具数据截掉。
- **修复**：上下文 = RAG + 全部 ToolMessage。
- **遗留**：截断策略仍需按来源分配（judge v2）。

故事 5：S002 的"安全 vs 可用性"

- **现象**：用户要查 1003 的记录，模型直接拒绝；用例期望"降级用当前账户 1001 取数"。
- **判断**：两种都安全，属产品口径问题，不该偷偷改用例。
- **做法**：保留失败、写进"已知问题"，等产品定义。

四、还没解决的（诚实清单）

问题	影响	计划
S002 口径冲突	通过率 -3.3pp	产品定义后改用例或改服务端策略
重排单价为 0	成本被低估	填官方单价后重跑
judge 三类偏差（报告/拒答/追问）	校准一致率仅 70%	judge v2：分类型 rubric + 按来源截断
忠实度跨版本口径变化	43.3→66.7 含修正量	以 v1.9 为新基线，同口径做 A/B
无"上下文管理"	长对话 token 上涨	P2：工具结果截断 + 历史摘要

五、你必须掌握的 12 个点

A. 必须能背（会被追问细节）

1. **两段式检索**：bi-encoder 召回（快、可预建索引）→ cross-encoder 精排（慢、准）。你的配置：`candidates=10 → topk=3`。
2. **BM25 三要素**：IDF（词稀有度）× TF 饱和（k1=1.5）× 长度归一（b=0.75）。
3. **RRF 公式**：`1/(k+rank)`，k=60；只用名次、免疫分数尺度。
4. **主指标定义**：hit@3（关键词覆盖）、chunk_recall@3（gold_chunk 是否在前 3）、chunk_MRR@5（首个正确块的倒数排名均值）。
5. **数字依据率**：回答里"带单位数字事实"在资料中出现的比例（规则、可解释）。
6. **判据版本 j5 的意义**：评分逻辑一改就必须升版本，否则跨版本分数不可比。

B. 能解释"为什么"

7. **为什么 judge 要看实际上下文**：否则它评的是"另一次检索"，不是模型真正看到的证据。
8. **为什么后处理只删数字句**：数字最可验证、误杀风险最低；改数字会制造新错误。
9. **为什么要 `--no-llm`**：检索层秒级、不烧对话额度，改 chunk/rerank 时可以高频回归。
10. **为什么要 `--compare`**：新基线要能"写进去"，同时还能对比历史基线。

C. 知道有这回事（能接话）

11. **judge 的偏差来源**：截断、rubric 缺失（拒答怎么打分）、模型自身偏好。
12. **S002 的两种安全策略**：拒绝 vs 降级取本人数据 —— 属产品决策。

六、面试问答（20 题，答法要点）

Q1 你这套评测解决什么问题？ A：把"感觉变好了"变成可复现的数字：30 条六类用例 + 四层指标（生成/检索/工具/工程）+ 分层归因 + 基线回归。

Q2 指标怎么定义？ A：hit@3 = top3 块的并集是否覆盖全部关键事实；chunk_recall@3 = 关键句是否落在 top3；chunk_MRR@5 = 首个正确块排名倒数均值；通过率 = 工具/参数/事实/拒答/追问全部通过。

Q3 为什么有 chunk 级还要文件级？ A：文件级（src@k）只能说明"找对文件"，同一文件很多块；块级才能定位"排序问题"。v1.7 就是靠它发现 recall@5=100 但 @3=87.5%。

Q4 重排为什么有效？ A：cross-encoder 把 query+doc 拼起来看词级交互，能纠正 bi-encoder"各看各的"的粗排错误；代价是每对都要算，所以只能对少量候选做。

Q5 BM25 为什么在你这里不如向量？ A：语料只有 ~40 块，IDF 退化、query 短，词频信号弱；而且中文要分词。修正 bug 后 BM25 能到 100% recall，但 MRR 仍低于 cross-encoder。

Q6 你踩过最大的坑？ A：BM25 实验的"候选序号 vs 全库序号"错位，让我把一个算法误判成"无效"。修完结论反转 —— 教训是**实验代码本身也要被测。**

Q7 judge 可靠吗？ A：不完全。我做了 20 条人工校准：相关性 ±1 内 85%，但忠实度一致率 70%、Kappa 0（报告/拒答类系统性偏严）。所以规则指标与 judge 并用。

Q8 忠实度怎么量? A：两条腿：judge（语义，看是否编造）+ 规则 `number_support_rate`（数字是否在资料里）。规则可解释、零成本，适合回归。

Q9 怎么抑制幻觉? A：三层：提示词（逐数字自检）、后处理（无依据数字删句）、评测（数字依据率回归）。实测 89.6%。

Q10 后处理会不会误杀? A：会，所以只处理"带单位的数字事实"，并且按句内事实判定；单测里专门有 `2-3 年 vs 2XL` 的回归用例。

Q11 你怎么保证可复现? A：基线里写 model / dataset_sha256 / judge_version / git_commit；温度 0；`--no-llm` 只跑检索。

Q12 跨版本比较注意什么? A：三条纪律：跨模型、跨判据、跨数据集都不可横比。v1.9 的忠实度还多了"上下文口径"变化。

Q13 为什么有 96.7% 不是 100%? A：唯一失败是 S002 越权用例：模型选择拒绝，用例期望"降级用本人 ID 取数"。这是产品口径，不是模型能力。

Q14 延迟怎么归因? A：埋点把重排耗时单列（均值 0.369s，降级 0 次）；P95 从 13.79→11.41 主要是模型调用变短，不是重排变快。

Q15 成本怎么算? A：tokens × 单价（入/出分开），基线记 `price`；重排按次/按量单列（当前未计价，报告会标注）。

Q16 异常和降级怎么处理? A：所有抛出点 `logger.error(exc_info=True)`；降级路径必打 WARNING（重排失败退回向量序、guard 失败返回原文、报告提示词失败退回客服提示词、agent 流失败给兜底回复）。

Q17 你的评测有什么局限? A：30 条样本小、judge 偏差、重排成本未计价、没有线上回流。下一步是 judge v2 + bad case 回流。

Q18 如果让你扩到 300 条? A：按真实问题分布分层抽样（检索/工具/拒答/多轮/安全），先做"高置信度子集"做门禁，长尾做监控。

Q19 怎么防上线回归? A：基线 JSON + 阈值（低于基线 X pp 就 `exit 1`），接 CI；`--compare` 直接给 delta。

Q20 这套东西最难的部分? A：不是写脚本，是**定义"什么算对"**：判据版本化、口径对齐（judge 看实际上下文）、以及承认"这次提升有多少是口径修正"。

七、速查

命令

```
set PYTHONPATH=%CD%

:: 检索层（秒级，不烧对话额度）
python eval\run_eval.py --no-llm
python eval\run_eval.py --no-llm --no-rerank          :: 纯向量对照

:: 全量 + judge（约 12 分钟）
python eval\run_eval.py --judge --judge-provider deepseek --judge-model deepseek-reasoner `
  --save-baseline --baseline eval\baseline_v19.json --compare eval\baseline_deepseek_v4.json

:: 重排对照（无 LLM）
python eval\exp_rerank_compare.py

:: judge 校准
python eval\judge_calibration.py --results eval\results\<run>.jsonl

:: 后处理单测
python eval\tests\test_guard.py
```

文件地图

想改什么	看哪个文件
检索/重排	`rag/vector_store.py`、`rag/reranker.py`、`config/chroma.yml`
RAG 回答与后处理	`rag/rag_service.py`、`rag/faithfulness_guard.py`、`prompts/rag_summarize.txt`
Agent 编排	`agent/react_agent.py`、`agent/tools/*`、`prompts/main_prompt.txt`
评测	`eval/run_eval.py`、`eval/dataset.jsonl`、`eval/prices.yml`
结果与证据	`eval/results/*.jsonl`、`eval/baseline_v19.json`、`docs/评测报告.md §15`

本总结由 v1.9 实际跑批数据生成：30 条用例、4 次跑批、1 次重排对照、20 条人工校准。